

Classifications and Metrics



SEA611
SIT@KMUTT

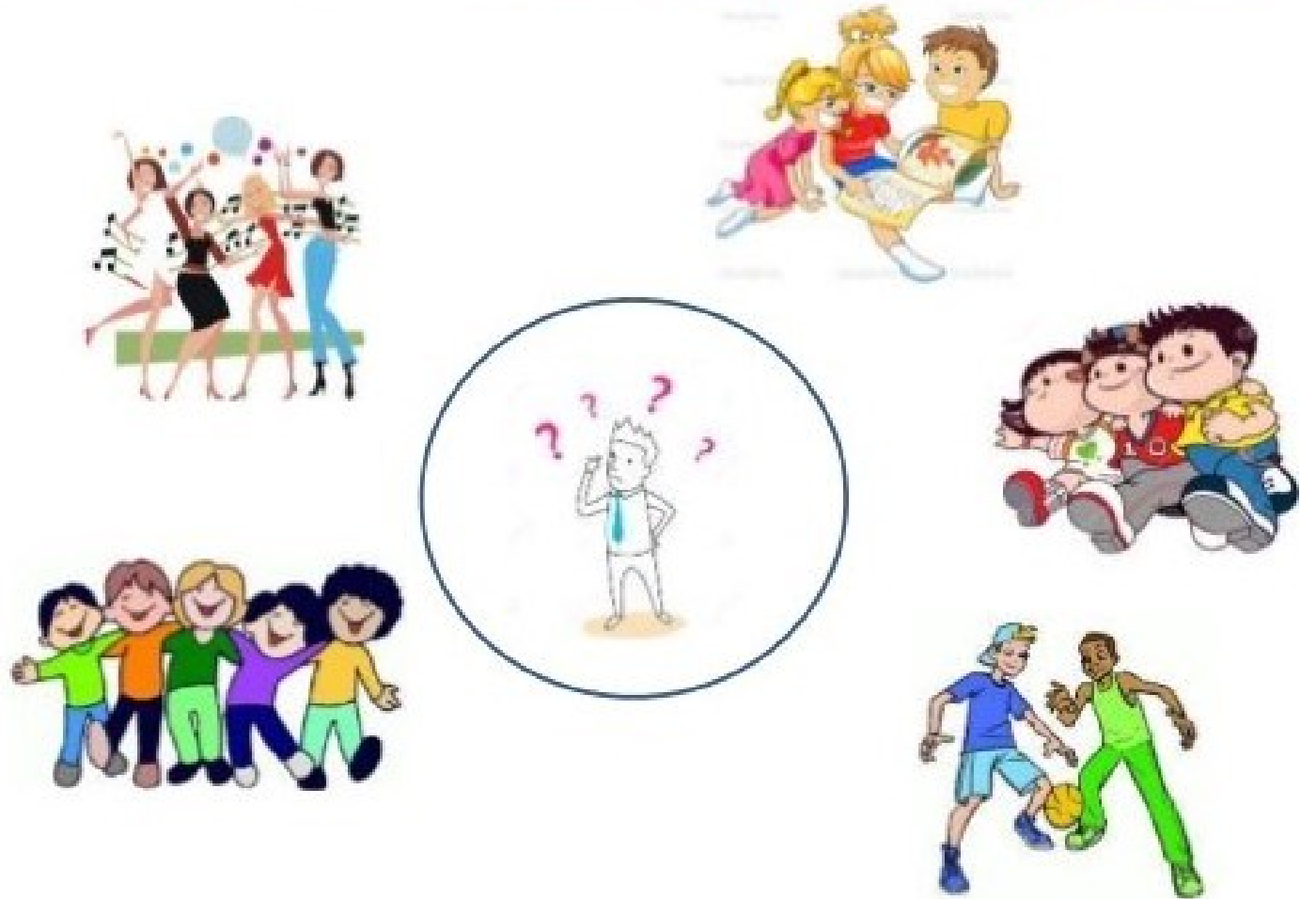
2026

Classification

Tell me about your friends(*who your neighbors are*) and *I will tell you who you are.*



Using different
classifiers



Classifiers

A **classifier** is a machine learning algorithm that is used to classify data into different categories. Classifiers are trained on a dataset of labeled data, which means that the data points in the dataset are already assigned to a category. The classifier learns from the labeled data and then uses this knowledge to classify new data points.

Classifiers are used in a wide variety of applications, including:

- **Image recognition:** Classifiers can be used to identify objects in images. For example, a classifier could be used to identify different types of flowers in a photo.
- **Natural language processing:** Classifiers can be used to classify text into different categories. For example, a classifier could be used to classify news articles into different categories, such as sports, business, or politics.
- **Fraud detection:** Classifiers can be used to detect fraudulent transactions. For example, a classifier could be used to identify credit card transactions that are likely to be fraudulent.
- **Medical diagnosis:** Classifiers can be used to diagnose diseases. For example, a classifier could be used to identify patients who are likely to have a certain type of cancer.

Classification methods

- In classification models, the **target variable** or class label is categorical, whereas in regression, the target variable is numeric
- Similar to regression, classification models are of two types: *parametric* and *nonparametric*
- **K-nearest neighbors** (KNN) makes predictions based on the data points from the training data that are in the neighborhood of the given test data point
- In **Decision Trees** (DT), we split the dataset at every node based on the value of a variable and classify the data points in a leaf node based on the aggregate values of the class labels
- **Naive Bayes** (NB) model classifies the data points using the data likelihood probabilities; the model makes the assumption of independence of features to simplify the model
- **Logistic regression** (LR), we fit the class probabilities using a sigmoid function and obtain the coefficients of this model using gradient descent
- **Support vector machine** (SVM) classifier learns the hyperplane that separates the data points between the two classes with the maximum margin, which is the closest distance between the data points of the two classes

K nearest neighbors (KNN): what is it?

K-nearest neighbors (KNN) is a non-parametric supervised learning algorithm that can be used for both classification and regression tasks. It works by finding the k most similar training examples to a new data point and then predicting the class label or value of the new data point based on the class labels or values of the k nearest neighbors.

The k value is a hyperparameter that needs to be chosen by the user. The value of k can affect the accuracy of the KNN algorithm. A larger value of k will make the KNN algorithm more robust to noise, but it may also make the algorithm less accurate. A smaller value of k will make the KNN algorithm more accurate, but it may also make the algorithm more sensitive to noise.

Critically, it then uses the **labels** of the K nearest neighbors to determine the new point's **classification (by a majority vote)** or **regression value (by an average)**. That's why it is considered supervised learning algorithm.

K nearest neighbors: pros & cons

Here are some of the advantages of using KNN:

- It is a simple and easy-to-understand algorithm.
- It is a versatile algorithm that can be used for a variety of tasks.
- It is relatively robust to noise.

Here are some of the disadvantages of using KNN:

- It can be computationally expensive for large datasets.
- It can be sensitive to the choice of k .
- It may not be as accurate as other machine learning algorithms for some tasks.

K nearest neighbors

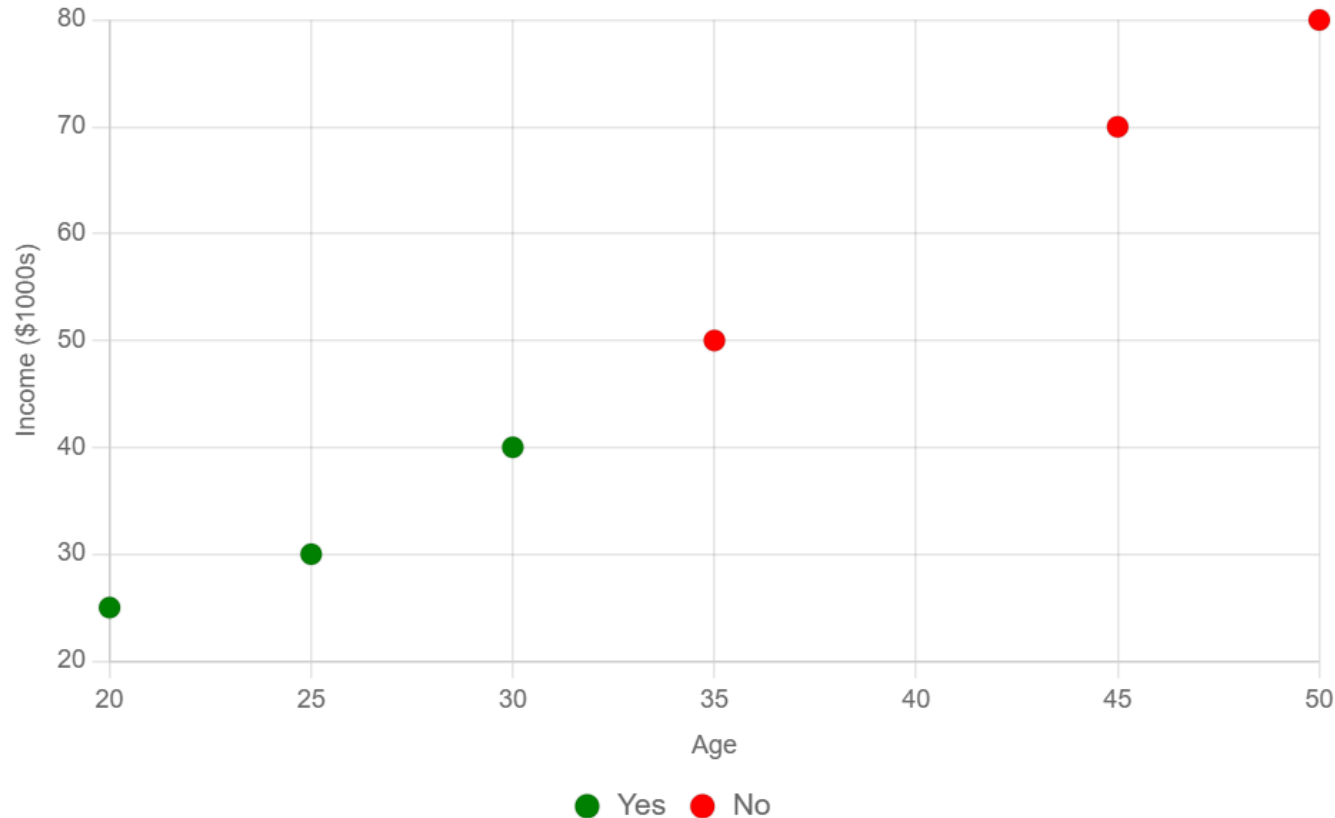
Problem: Predict if a person likes sports based on age and income

Training Data:

PERSON	AGE	INCOME (\$1000S)	LIKES SPORTS?
A	25	30	Yes
B	35	50	No
C	45	70	No
D	20	25	Yes
E	30	40	Yes
F	50	80	No

K nearest neighbors

KNN Training Data: Likes Sports?



New Data Point to Classify:

- **Person X:** Age = 28, Income = 35

KNN Process (K=3):

- **3 nearest neighbors:** E(Yes), A(Yes), D(Yes)
- **Majority vote:** $3/3 = 100\%$ Yes
- **Prediction:** Person X likes sports = YES

Decision tree (DT): what is it?

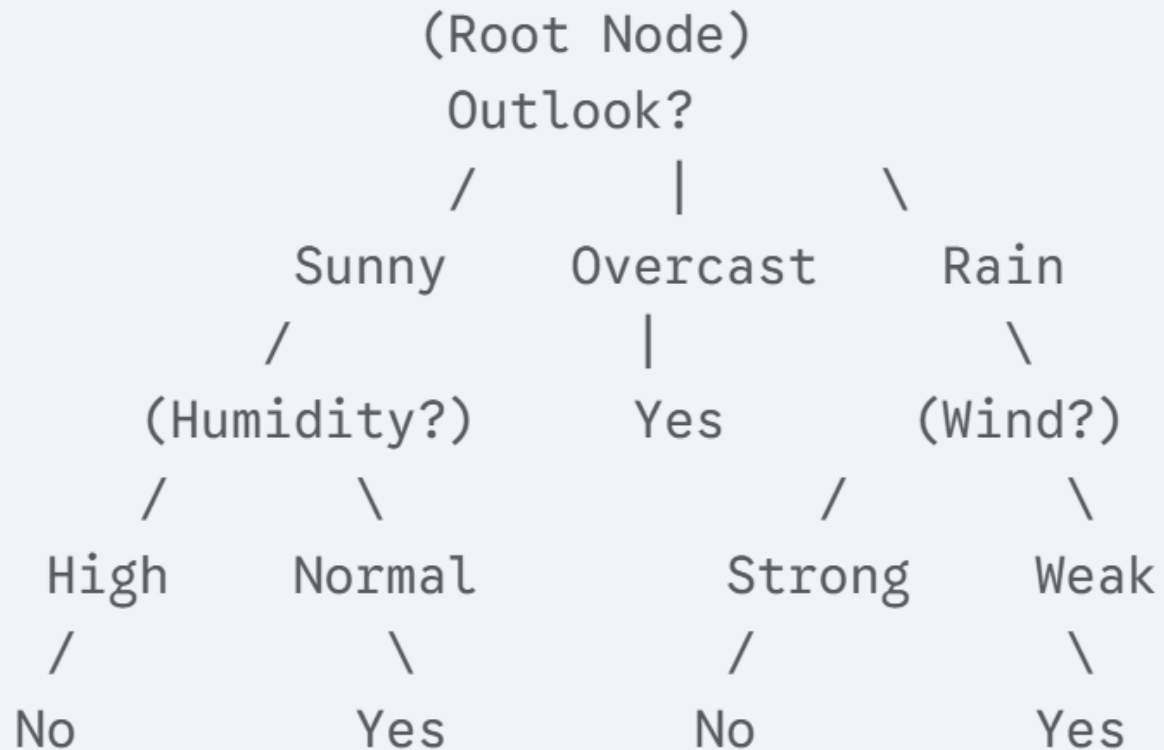
A decision tree is a supervised machine learning algorithm that can be used for both classification and regression tasks. It is a flowchart-like structure that uses a series of yes/no questions to arrive at a decision.

The decision tree is built by recursively splitting the training data into smaller and smaller subsets based on the values of the features. The splitting process is repeated until a stopping criterion is met, such as the maximum depth of the tree or the minimum number of samples required to split a node.

The leaves of the decision tree represent the decisions that can be made, and the branches represent the questions that need to be answered to reach those decisions. The decision tree can be used to predict the class label of a new data point by following the branches of the tree until it reaches a leaf node.

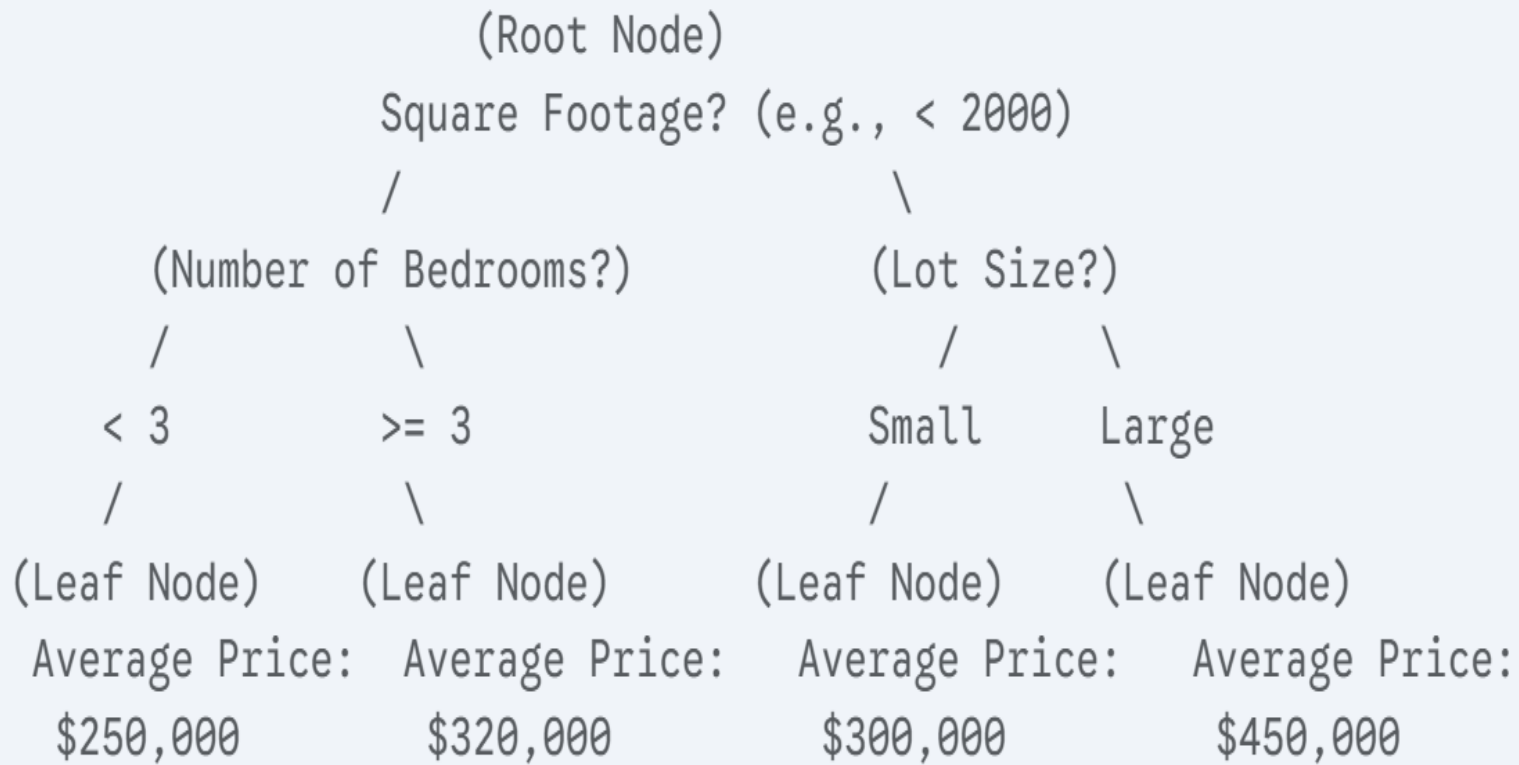
Decision tree for classification

Play an outdoor sport?



This example shows a basic decision tree where it is to be decided in what conditions to **play an outdoor sport** and in what conditions not to play.

Decision tree for regression



Predict a house price

 **House A:** 1,800 sq.ft, 2 bedrooms → **\$250,000**

- Square Footage < 2000? ✓ → Check Bedrooms
- Bedrooms < 3? ✓ → **\$250K**

 **House B:** 2,500 sq.ft, Large lot → **\$450,000**

- Square Footage < 2000? × → Check Lot Size
- Lot Size = Large? ✓ → **\$450K**

Decision tree: find the best root using Gini Impurity

Problem: Imagine you're trying to build a decision tree to predict if a house's price is "High" or "Low." You have data for several houses, and for each house, you know a few things about it.

Here are your features (the questions you can ask):

- **Square Footage:** Is it greater than 2,000 sq ft? (Yes/No)
- **Number of Bathrooms:** Is it 2 or more? (Yes/No)
- **Neighborhood:** Is it in a "Good" neighborhood? (Yes/No)

You have a dataset of 10 houses. Let's say out of those 10 houses:

- 5 have a "High" price.
- 5 have a "Low" price.

Gini impurity is a way of measuring how "mixed up" or "impure" a group of data is. The Gini impurity value ranges from **0 to 0.5** (for a two-class problem, or higher for more classes). A value 0 means the group is perfectly **pure**. You can't make a better split than this.

Decision tree: find the best root using Gini Impurity

Let's try to use "Neighborhood" as the Root Node

- **Split 1: Houses in a "Good" neighborhood**
 - You find 5 houses in this group.
 - 4 of them have a "High" price.
 - 1 of them has a "Low" price.
 - This group is **very pure**, leaning heavily toward "High" prices.
- **Split 2: Houses NOT in a "Good" neighborhood**
 - You find 5 houses in this group.
 - 1 of them has a "High" price.
 - 4 of them have a "Low" price.
 - This group is **also very pure**, leaning heavily toward "Low" prices.

Decision tree: find the best root using Gini Impurity

The formula for Gini impurity is:

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

Where:

- C is the number of classes (e.g., "High" price and "Low" price). 
- p_i is the probability of an item belonging to class i . This is simply the count of items in class i divided by the total number of items in the group.

The goal isn't just to calculate the Gini impurity of a single group. The decision tree algorithm calculates the **weighted average Gini impurity** for the *resulting branches* after a potential split. It then chooses the split that results in the **lowest weighted Gini impurity**.

Decision tree: find the best root using Gini Impurity

Let's revisit our "Neighborhood" example:

- **Split 1: Houses in a "Good" neighborhood**
 - 5 houses: 4 "High" and 1 "Low."
 - $p_{\text{High}} = 4/5 = 0.8$
 - $p_{\text{Low}} = 1/5 = 0.2$
 - $Gini_{\text{Good}} = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32$
- **Split 2: Houses NOT in a "Good" neighborhood**
 - 5 houses: 1 "High" and 4 "Low."
 - $p_{\text{High}} = 1/5 = 0.2$
 - $p_{\text{Low}} = 4/5 = 0.8$
 - $Gini_{\text{Not Good}} = 1 - (0.2^2 + 0.8^2) = 1 - (0.04 + 0.64) = 1 - 0.68 = 0.32$

Decision tree: find the best root using Gini Impurity

Now, we calculate the weighted average of these two Gini values. Since each branch has 5 houses out of the total 10, the weight for each is $5/10 = 0.5$.

- **Weighted Gini Impurity for "Neighborhood" split:**
 - $(0.5 \times Gini_{\text{Good}}) + (0.5 \times Gini_{\text{Not Good}})$
 - $(0.5 \times 0.32) + (0.5 \times 0.32) = 0.16 + 0.16 = 0.32$

The Gini impurity for the entire split is 0.32, which is a significant reduction from the original 0.5.

Decision tree: pros & cons

Here are some of the advantages of using decision trees:

- They are relatively easy to understand and interpret.
- They can be used with both numerical and categorical data.
- They can be used to solve a variety of problems.

Here are some of the disadvantages of using decision trees:

- They can be sensitive to overfitting.
- They can be difficult to scale to large datasets.
- They can be brittle, meaning that they can be easily affected by changes in the data.

Naïve Bayes (NB): what is it?

Naive Bayes assumes that the features of a data point are independent of each other. This means that the probability of a data point belonging to a particular class is the product of the probabilities of each of its features belonging to that class.

For example, let's say we have a dataset of emails, and we want to classify them as spam or not spam. We can use Naive Bayes to do this by assuming that the features of an email, such as the sender, the subject line, and the body of the email, are independent of each other. This means that the probability of an email being spam is the product of the probabilities of the sender being a spammer, the subject line being spammy, and the body of the email being spammy.

Naive Bayes is a relatively simple algorithm, but it can be very effective for classification tasks. It is also relatively fast and easy to train, which makes it a good choice for large datasets.

Naïve Bayes

The diagram shows the Naïve Bayes formula with arrows pointing from descriptive labels to the corresponding terms in the equation:

$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Labels and their corresponding terms:

- Likelihood** points to $P(x | c)$
- Class Prior Probability** points to $P(c)$
- Posterior Probability** points to $P(c | x)$
- Predictor Prior Probability** points to $P(x)$

$$P(c | \mathbf{X}) = P(x_1 | c) \times P(x_2 | c) \times \cdots \times P(x_n | c) \times P(c)$$

- $P(c|x)$ is the posterior probability of *class (target)* given *predictor (attribute)*.
- $P(c)$ is the prior probability of *class*.
- $P(x|c)$ is the likelihood which is the probability of *predictor* given *class*.
- $P(x)$ is the prior probability of *predictor*.

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

$$P(\text{Spam}|\text{Words}) = \frac{P(\text{Words}|\text{Spam}) \times P(\text{Spam})}{P(\text{Words})}$$

In simpler terms, we are trying to find:

- $P(\text{Spam}|\text{Words})$: The probability that an email is spam, **given** the words it contains.

To find this, we calculate:

- $P(\text{Spam})$: The **prior probability** that any email is spam (e.g., if 50% of your emails are spam, this is 0.5).
- $P(\text{Words}|\text{Spam})$: The probability of seeing these specific words, **given** that the email is spam.
- $P(\text{Words})$: The overall probability of seeing these words.

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

For classification, we don't need to calculate $P(\text{Words})$ because it's the same for both spam and ham. We simply compare the numerator for both classes.

We'll compare:

1. $P(\text{Spam}|\text{Words}) \propto P(\text{Words}|\text{Spam}) \times P(\text{Spam})$
2. $P(\text{Ham}|\text{Words}) \propto P(\text{Words}|\text{Ham}) \times P(\text{Ham})$

The email is classified as the class with the higher probability.

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

Training Data:

Email Text	Class
Email 1: "Dear friend, money"	Spam
Email 2: "Free money now"	Spam
Email 3: "Urgent, claim your prize"	Spam
Email 4: "Free money today"	Spam
Email 5: "My dear friend"	Ham
Email 6: "Dear friend, how are you?"	Ham
Email 7: "Let's meet today"	Ham
Email 8: "Claim your prize"	Ham
Email 9: "Urgent, reply soon"	Ham
Email 10: "How are you today"	Ham

Word	Count in Spam	Count in Ham
friend	2	2
money	3	0
free	2	0
urgent	1	1
claim	1	1
prize	1	1
dear	1	2
today	1	2
...	...	...
Total words	10	14

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

- $P(\text{Spam})$: There are 4 spam emails out of 10. So, $P(\text{Spam}) = \frac{4}{10} = 0.4$
- $P(\text{Ham})$: There are 6 ham emails out of 10. So, $P(\text{Ham}) = \frac{6}{10} = 0.6$
- $P(\text{money}|\text{Spam}) = \frac{\text{Count of 'money' in Spam}}{\text{Total words in Spam}} = \frac{3}{10} = 0.3$
- $P(\text{money}|\text{Ham}) = \frac{\text{Count of 'money' in Ham}}{\text{Total words in Ham}} = \frac{0}{14} = 0$
- $P(\text{dear}|\text{Spam}) = \frac{1}{10} = 0.1$
- $P(\text{dear}|\text{Ham}) = \frac{2}{14} \approx 0.143$
- $P(\text{friend}|\text{Spam}) = \frac{2}{10} = 0.2$
- $P(\text{friend}|\text{Ham}) = \frac{2}{14} \approx 0.143$

Let's classify a new email: **"Dear friend, free money"**

We need to calculate two probabilities:

1. The probability of this email being spam.
2. The probability of this email being ham.

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

1. Probability of being Spam:

We assume the words are independent, so we multiply their probabilities together.

$$P(\text{Spam} | \text{"Dear friend free money"}) \propto P(\text{dear} | \text{Spam}) \times P(\text{friend} | \text{Spam}) \times P(\text{free} | \text{Spam}) \times P(\text{money} | \text{Spam}) \times P(\text{Spam})$$

- $P(\text{dear} | \text{Spam}) = 0.1$
- $P(\text{friend} | \text{Spam}) = 0.2$
- $P(\text{free} | \text{Spam}) = 0.2$
- $P(\text{money} | \text{Spam}) = 0.3$
- $P(\text{Spam}) = 0.4$

$$\text{Spam Probability Score} = 0.1 \times 0.2 \times 0.2 \times 0.3 \times 0.4 = 0.00048$$

Naïve Bayes for classifies emails as "spam" or "not spam or ham)

2. Probability of being Ham:

$$P(\text{Ham} | \text{"Dear friend free money"}) \propto P(\text{dear} | \text{Ham}) \times P(\text{friend} | \text{Ham}) \times P(\text{free} | \text{Ham}) \times P(\text{money} | \text{Ham}) \times P(\text{Ham})$$

- $P(\text{dear} | \text{Ham}) \approx 0.143$
- $P(\text{friend} | \text{Ham}) \approx 0.143$
- $P(\text{free} | \text{Ham}) = 0$
- $P(\text{money} | \text{Ham}) = 0$
- $P(\text{Ham}) = 0.6$

$$\text{Ham Probability Score} = 0.143 \times 0.143 \times 0 \times 0 \times 0.6 = 0$$

Conclusion:

Since the Spam Probability Score (0.00048) is much greater than the Ham Probability Score (0), the Naive Bayes algorithm would confidently classify the email **"Dear friend, free money"** as **Spam**.

Naïve Bayes: pros & cons

Here are some of the advantages of using Naive Bayes:

- It is relatively simple and easy to understand.
- It is fast and easy to train.
- It can be effective for a variety of classification tasks.

Here are some of the disadvantages of using Naive Bayes:

- It assumes that the features of a data point are independent of each other. This assumption may not be true in all cases.
- It can be sensitive to the presence of outliers.
- It may not be as accurate as other machine learning algorithms for some tasks.

Logistic regression: what is it?

Logistic regression is a statistical model that is used to predict the probability of a binary outcome, such as whether a customer will click on an ad or not. It is a type of regression analysis, but instead of predicting a continuous value, it predicts the probability of a binary outcome.

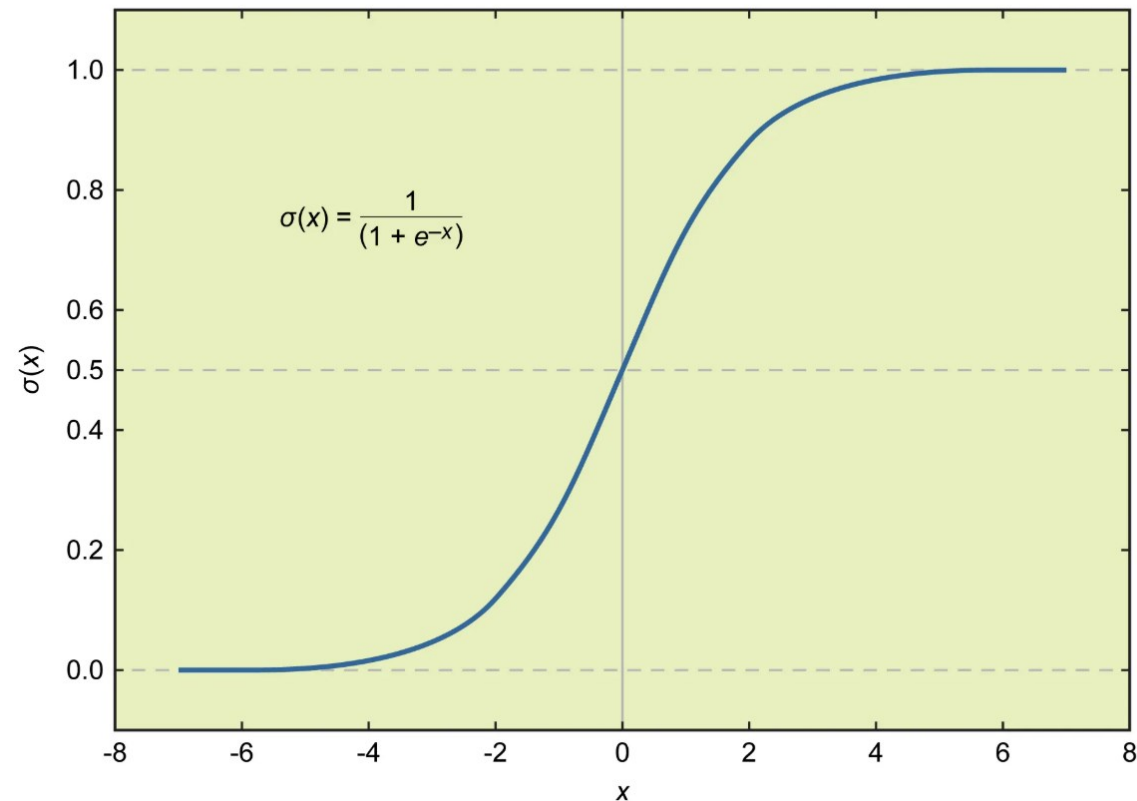
Logistic regression works by fitting a logistic function to the data. The logistic function is a sigmoid function that maps the real number line to the interval $[0, 1]$. This means that the logistic function can be used to represent the probability of a binary outcome.

Logistic regression is a powerful tool for predicting binary outcomes. It is relatively easy to understand and interpret, and it can be used with both numerical and categorical data. However, logistic regression can be sensitive to overfitting, and it can be difficult to scale to large datasets.

In addition to **numerical data**, the logistic regression can **use categorical data**, but you must first transform it into a numerical representation, with **one-hot encoding**.

Logistic (sigmoid) function

Sigmoid function



The logistic function that is a sigmoid function is an **S shaped curve** that takes any real values and converts them between 0 to 1, considered as **probability [0, 1]**.

If the output given by a sigmoid function is more than 0.5, the output is classified as 1, but if it is less than 0.5, the output is classified as 0, considered as **binary classification**.

Logistic regression: Example of website clicks

Customer	Time on Site (minutes)	Clicked? (1 = Yes, 0 = No)
1	0.5	0
2	1.0	0
3	1.2	0
4	2.5	0
5	3.0	1
6	4.0	1
7	4.5	1
8	5.5	1
9	6.0	1
10	7.0	1

Problem: If a new customer spends **3.5 minutes** on a website, will she click a 'Buy' button?

Probability of Click = $f(\text{Time on Site})$, where f is a sigmoid function.

Logistic regression: Example of website clicks

$$P(\text{Click} = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \cdot (\text{Time on Site}))}}$$

The logistic regression algorithm's job during training is to learn the best values for β_0 and β_1 (and any other β coefficients if you have more features) that best fit your training data and allow the S-shaped curve to accurately represent the relationship between your features and the probability of the outcome.

The last step is to turn this probability into a final classification (Yes or No). You need to set a **threshold**. The standard threshold is **0.5**.

- If **Probability > 0.5**, the model predicts **Yes** (they will click).
- If **Probability < 0.5**, the model predicts **No** (they won't click).

Logistic regression: pros & cons

Here are some of the advantages of using logistic regression:

- It is relatively easy to understand and interpret.
- It can be used with both numerical and categorical data.
- It is a powerful tool for predicting binary outcomes.

Here are some of the disadvantages of using logistic regression:

- It can be sensitive to overfitting.
- It can be difficult to scale to large datasets.
- It may not be as accurate as other machine learning algorithms for some tasks.

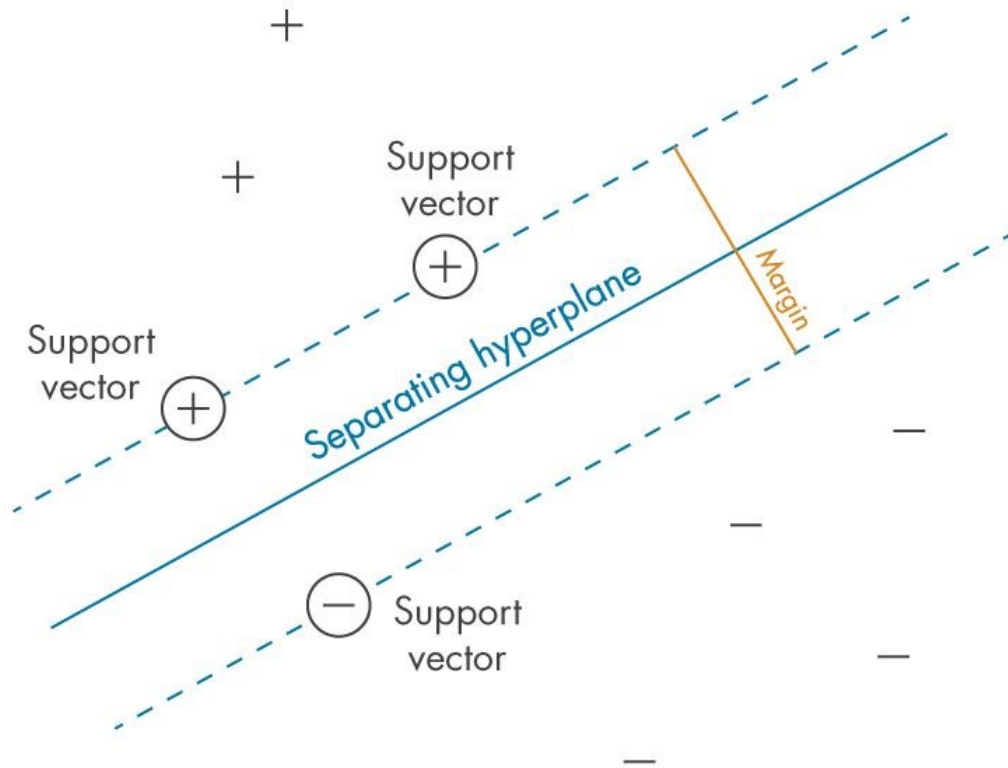
Support vector machine: what is it?

A support vector machine (SVM) is a supervised machine learning algorithm that can be used for both classification and regression tasks. It is a discriminative classifier, which means that it tries to find a decision boundary that separates the two classes of data.

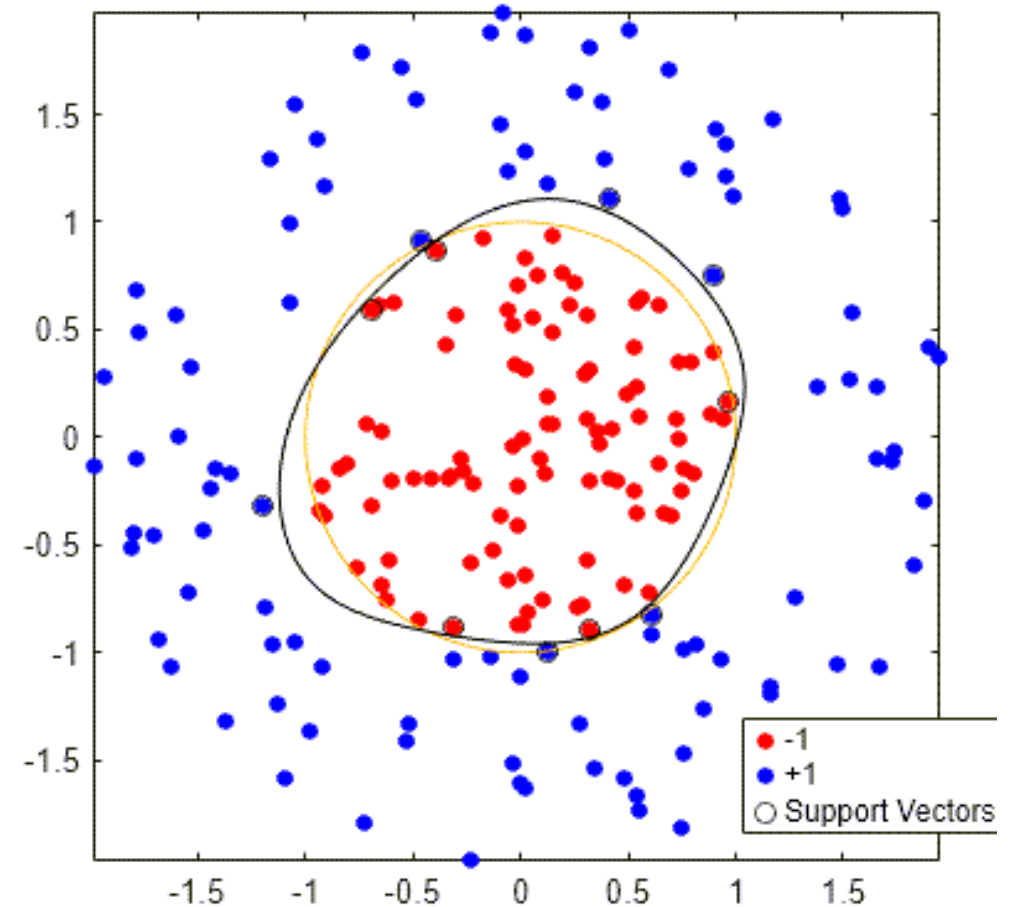
SVM works by finding the hyperplane that best separates the two classes of data. The hyperplane is a line or a plane in n-dimensional space that divides the data into two classes. The SVM algorithm tries to find the hyperplane that has the maximum margin, which is the distance between the hyperplane and the closest data points of each class.

The support vectors are the data points that are closest to the hyperplane. These points are the ones that influence the position and orientation of the hyperplane. The SVM algorithm tries to find a hyperplane that maximizes the margin and also minimizes the number of support vectors.

SVM terminology



<https://www.mathworks.com/>



A nonlinear SVM classifier with Gaussian kernel function.

Note: If the data is non linearly separable as shown in the above figure then SVM makes use of kernel tricks to make it linearly separable. Kernel tricks help in projecting data points to the higher dimensional space by which they became relatively more easily separable in higher-dimensional space.

Support vector machine: pros & cons

Here are some of the advantages of using SVM:

- It is a powerful machine learning algorithm that can be used for a variety of tasks.
- It is relatively robust to noise and outliers.
- It can be used for both classification and regression tasks.

Here are some of the disadvantages of using SVM:

- It can be computationally expensive for large datasets.
- It can be sensitive to the choice of kernel function.
- It may not be as accurate as other machine learning algorithms for some tasks.

Classification

- A classification model is fitted to *classify* data points into multiple categories or *classes*
- Target variable is referred as the *class label*
- Classification models with target variables taking more than two values is called *multi-class* classification, and *binary* classification if two values
- E.g., spam email filtering, speech and handwriting recognition, and biometric authentication
- Model types:
 - **Parametric**: Naive Bayes (NB), logistic regression, support vector machine (SVM)
 - **Non-parametric**: K-nearest neighbors (KNN), decision tree-based models

Remark: Statistical models are of two types: **parametric**, where we assume a functional relationship between the predictor and target variables, and **non-parametric**, where we do not assume any such relationship.

Binary Classification vs. Multi-class Classification

We can use a binary classification algorithm for multi-classification problems to split the multi-class classification dataset into multiple binary classification datasets and fit a binary classification model on each.

Two approaches:

- One-vs-Rest (OvR), e.g., as can be used in logistic regression
- One-vs-One (OvO), e.g., as can be used in SVM

OvR approach

Let's be given a multi-class classification problem with examples for each class '**red**', '**green**', '**blue**', and '**yellow**', we will have the following problems:

- ✓ Binary Classification Problem 1: **red** vs [green, blue, yellow]
- ✓ Binary Classification Problem 2: **green** vs [red, blue, yellow]
- ✓ Binary Classification Problem 3: **blue** vs [red, green, yellow]
- ✓ Binary Classification Problem 4: **yellow** vs [red, green, blue]

We split the multi-class dataset into multiple binary classification problems. A binary classifier is then trained on each binary classification problem, and predictions are made using the model that is the most confident (e.g., probability).

OvO approach

Let's consider a multi-class classification problem with four classes: **'red,'** **'green,'** **'blue,'** and **'yellow.'**

This could be divided into six binary classification datasets as follows:

- ✓ Binary Classification Problem 1: red vs. blue
- ✓ Binary Classification Problem 2: red vs. green
- ✓ Binary Classification Problem 3: red vs. yellow
- ✓ Binary Classification Problem 4: blue vs. green
- ✓ Binary Classification Problem 5: blue vs. yellow
- ✓ Binary Classification Problem 6: green vs. yellow

Each binary classification model may predict one class label, and the model with the most predictions or votes is predicted.

Multi-label classifications

Multi-label classification is a type of supervised machine learning algorithm that can be used to assign **zero or more labels** to each data sample. E.g., a multi-label classifier could be used to classify an image that consists of many animals such as a dog, fowl, donkey, and a cat.

The multi-label classification is often used in image recognition tasks.



Brief summary about classification types

- Binary Classification is where each data sample is assigned one and only one label from two mutually exclusive classes.
- Multi-class Classification is where each data sample is assigned one and only one label from more than two classes.
- Multi-label Classification is a type of supervised machine learning algorithm that can be used to assign zero or more labels to each data sample.

Data: Training Set and Test Set

- We split the dataset into *training* and *test* datasets
- Use the training data to fit the model and test data to evaluate it.
- It is generally not a good idea to evaluate a model on the same data it is trained on due to possible biases
- Having a model that fits the training data with very high accuracy indicates that it is also fitted to some of the biases in that dataset
- Our goal is to obtain a model that is able to predict new data points.
- However, in the process of optimizing our model to minimize the classification error on test data, we might fit to the biases in the dataset
- Techniques such as *cross-validation* are designed to solve this problem, with the use of *validation* dataset, which is used to tune hyperparameters of the model; thereafter, the test dataset is used to test the model. If the test dataset has never been used in training (e.g., in cross-validation), it is also called a *holdout* dataset

One-time doing train, validate, and test of dataset

- Step 1: Training dataset; e.g. 70% split of the dataset
 - The sample of data used to *fit* the model
 - The model *sees* and *learns* from this data
- Step 2: Validation dataset; e.g., 10% split of the dataset
 - The sample of data is used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters
 - The model occasionally *sees* this data, but it does not *learn* from it
 - Adjust some hyperparameters, then repeat Step 1 and Step 2 several times until a validation performance stops improving. We stop training **at the point where the validation loss was at its absolute lowest value**, even if the training loss is still going down.
 - Note: The validation set is also known as the Dev set or the development set
- Final step: Test dataset; e.g., 20% split of the dataset
 - The sample of data is used to provide an unbiased *evaluation* of a final model fit on the training dataset
 - It is only run one time after a model is completely trained (i.e., after using the train and validation sets)



Train, validate, and test

Two approaches we can do:

1. Train-Validate-Test split, say, 70:20:10 percentage

- If our data is huge and our test sample and train sample have the same distribution then this approach is acceptable

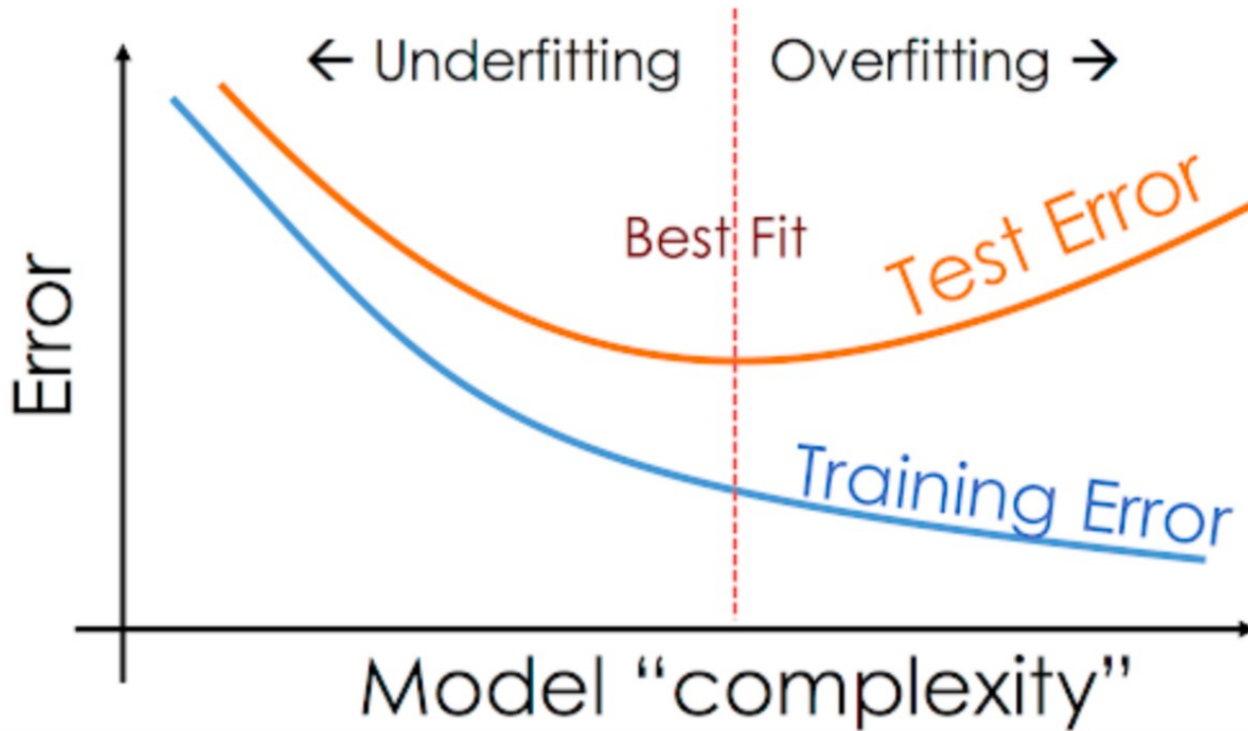
2. K-folds cross validation

- It generally results in a less biased model compare to other methods. Because it ensures that every observation from the original dataset has the chance of appearing in training and validation set.
- K is preferably set to 5-10 depending on the data size. The higher value of K leads to less biased model (but large variance might lead to overfit), whereas the lower value of K is similar to the Train-Test split approach.
- Set some hyperparameters, then train the K-1 folds of data, then validate with the remaining 1 fold.
- Compute an average accuracy of the K folds; repeat the previous steps with another set of hyperparameters.

To reduce overfitting problem: do **K-fold cross validation**

1. Let's assume the use of 5-fold cross validation
2. First, split the dataset into 2 groups: *Train (e.g., 80%)* and *Test (20%)* datasets
3. Then keep aside the Test set for a final test.
4. Now split the 80% into 5 folds, set some hyperparameters, then do the full 5-fold cross validation. Try different set of hyperparameters as many rounds as needed, and record the average accuracy of each round.
5. After many rounds, select the hyperparameters (i.e, the best model) with the highest average accuracy score (from the best round).
6. Then use the best hyperparameters you just got to train from scratch on the Train 80% dataset one more time, and then you will get a final model.
7. No need to do any validation anymore.
8. Run the final model after training with the *Test (20%)* dataset that were hold out initially.
9. Last, obtain an accuracy of the final model.

Overfitting vs. Underfitting



- A model is said to be **underfitting** if both the training and the test error is sufficiently high.
 - A model is said to be **overfitting** if there is a large gap between training and test error with test error significantly higher than the training error.
 - An underfitting model tends to have high bias. An overfitting model tends to have high variance.
- A model is said to be biased (high bias) if the model accuracy is low. A model is said to have high variance if multiple models trained on the training data set derived from the same population make significantly different predictions given the input data set

Model evaluation: Confusion Matrix (also called error matrix)

- It is used to determine the performance of classifier
- It allows easy identification of confusion between classes e.g. one class is commonly mislabeled as the other
- The number of correct and incorrect predictions are summarized with count values and broken down by each class

- **True Positive (TP)** : Observation is positive, and is predicted to be positive.
- **False Negative (FN)** : Observation is positive, but is predicted negative.
- **True Negative (TN)** : Observation is negative, and is predicted to be negative.
- **False Positive (FP)** : Observation is negative, but is predicted positive.

	<i>Class 1 Predicted</i>	<i>Class 2 Predicted</i>
Class 1 Actual	TP	FN
Class 2 Actual	FP	TN

Summary of metrics

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Overview: some popular metrics

$$\textit{Recall} = \frac{TP}{TP + FN} \quad = \text{Sensitivity} = \text{True positive rate (TPR)} = \text{Hit rate}$$

$$\textit{Precision} = \frac{TP}{TP + FP}$$

$$\textit{Accuracy} = \frac{TP + TN}{\textit{TotalSamples}} = \frac{TP + TN}{TP + FP + TN + FN}$$

$$\textit{Specifcity} = \frac{TN}{FP + TN} \quad = \text{Selectivity} = \text{True negative rate (TNR)}$$

False negative rate (FNR) = 1 – True positive rate (TPR) = Miss rate

False positive rate (FPR) = 1– True negative rate (TNR) = Fall out

Multi-class Classification: Confusion Matrix

(Do it class by class)

Iris Flowers

		Predicted Values		
		Setosa	Versicolor	Virginica
Actual Values	Setosa	16 (cell 1)	0 (cell 2)	0 (cell 3)
	Versicolor	0 (cell 4)	17 (cell 5)	1 (cell 6)
	Virginica	0 (cell 7)	0 (cell 8)	11 (cell 9)

Micro vs. Macro Accuracy

Micro accuracy

→ combined all classes =
 $(16+17+11)/(16+17+11+1+0+0+0+0)$
 $= 44/45 = 0.9778$

Macro accuracy

→ average each class equally =
 $16/(16+0+0) +$
 $17/(17+0+1) +$
 $11/(11+0+0)$
 $= 1 + 0.94 + 1 = 2.94$

Macro accuracy = $2.94 / 3 = 0.9815$

Metrics: more details

$$\textbf{Precision} = \frac{TP}{TP + FP}$$

High Precision indicates an example labelled as positive is indeed positive (a small number of FP).

$$\textbf{Recall} = \frac{TP}{TP + FN}$$

→ also called **Sensitivity**
or **True Positive Rate**

Recall can be defined as the ratio of the total number of correctly classified positive examples divide to the total number of positive examples. High Recall indicates the class is correctly recognized (a small number of FN)

Metrics (cont'd)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Notice that there is a problem with accuracy. It assumes equal costs for both kinds of errors

$$\text{F - measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}} \quad \rightarrow \text{Also called } \mathbf{F1 \text{ score}}$$

It represents both recall and precision as it is a weighted average of the recall (sensitivity) and precision. The F-measure uses Harmonic Mean instead of Arithmetic Mean as it punishes the extreme values more. The F-Measure will always be nearer to the smaller value of Precision or Recall, and it might be a good choice when you seek to balance between Precision and Recall. The highest possible value is 1, indicating perfect precision and recall

Metrics (cont'd)

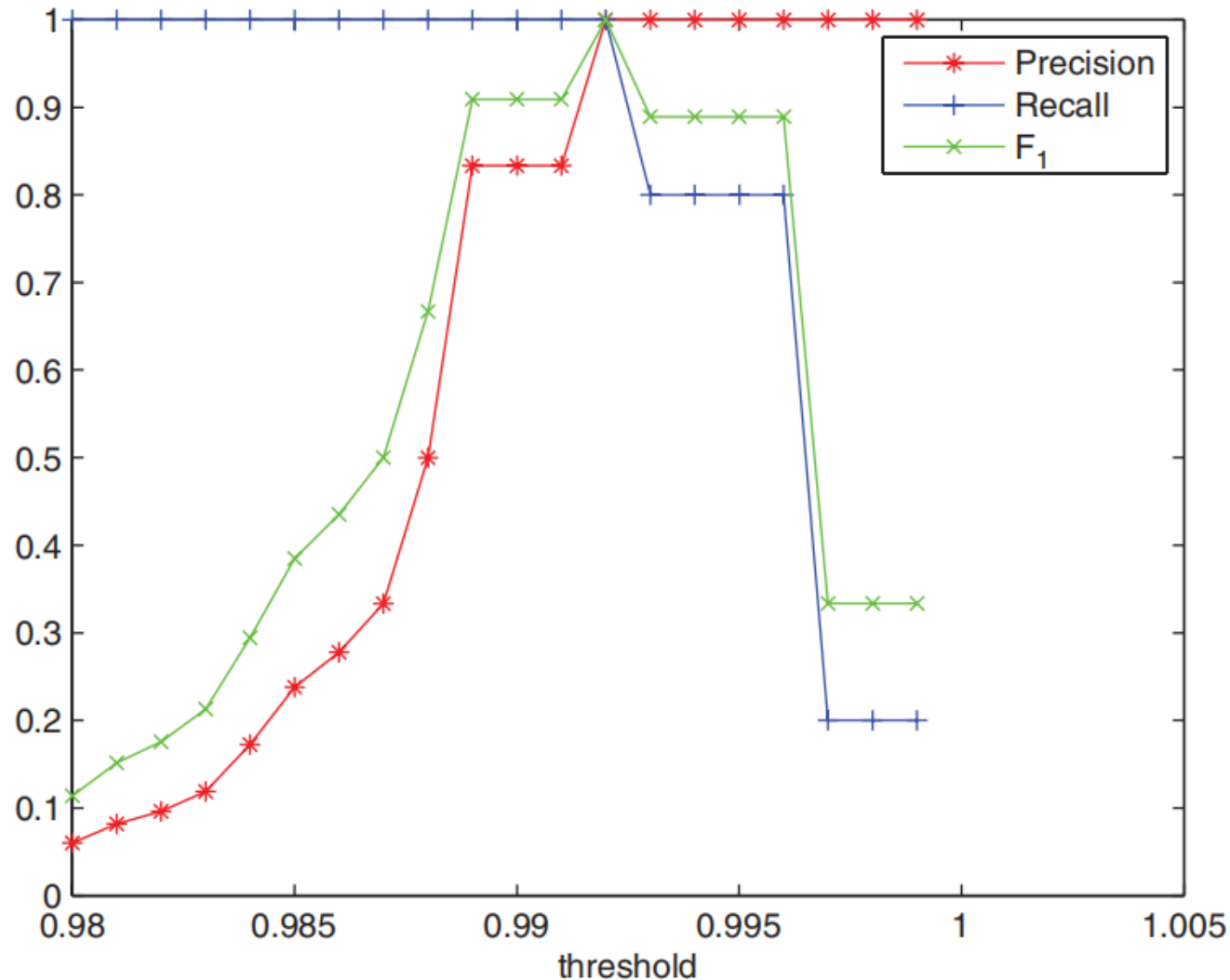
$$F_{\beta} = (1 + \beta^2) \cdot \frac{\textit{Precision} \cdot \textit{Recall}}{(\beta^2 \cdot \textit{Precision}) + \textit{Recall}}$$

The following are 3 commonly used **F-measures** (also called **F-score**):

- The F_1 measure puts even weight on precision and recall.
- The F_2 measure puts a higher weight on recall than precision.
- The $F_{0.5}$ measure puts a higher weight on precision than recall.

Note: Some stated that F was named after Ronald Fisher whose work was in F-distribution, F-test, etc.

Example: Performance metrics graph

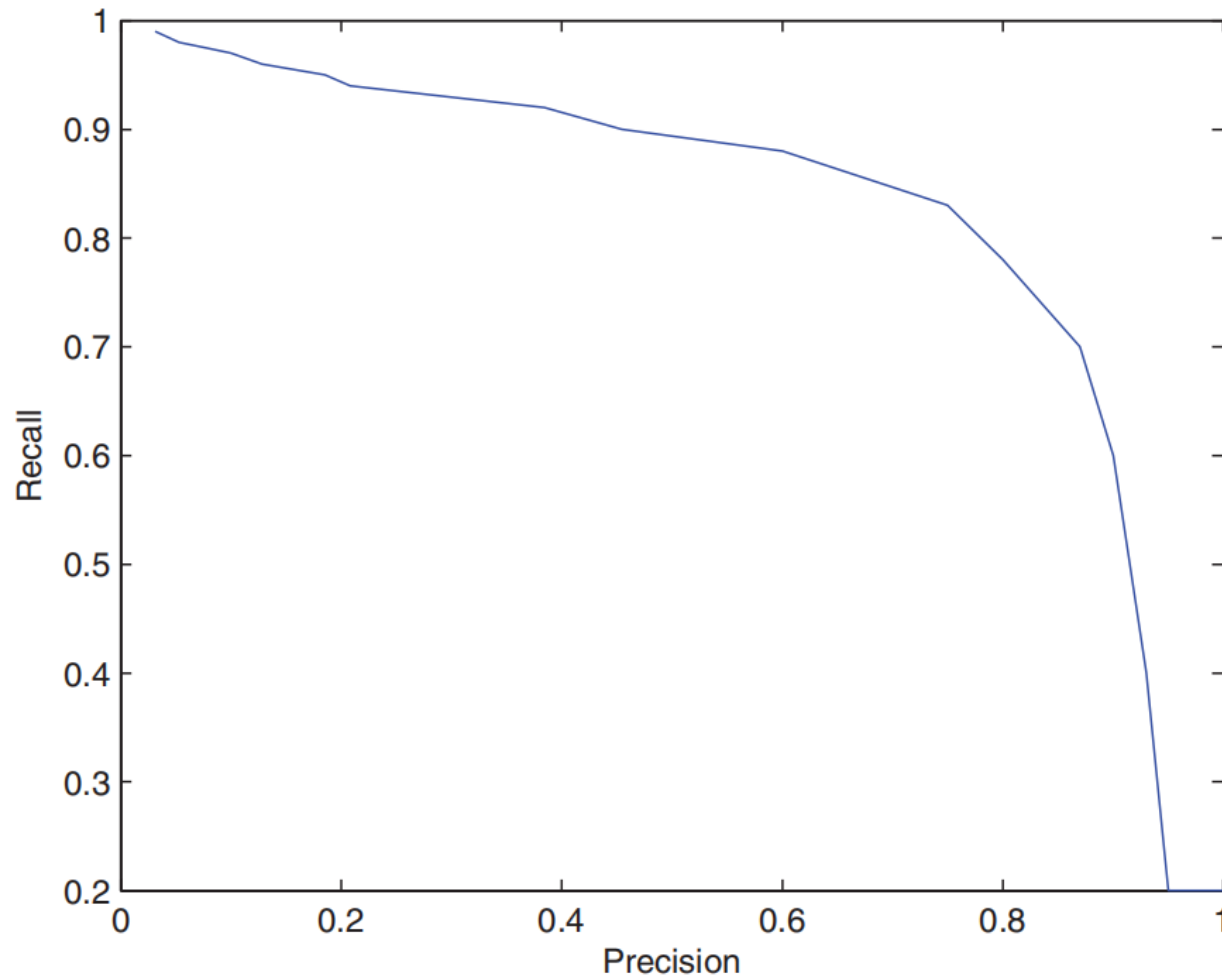


Note:

The specificity and accuracy values are not shown as they are both approximately 1.0 for all values of the threshold.

A threshold value of 0.992 would have given a perfect result.

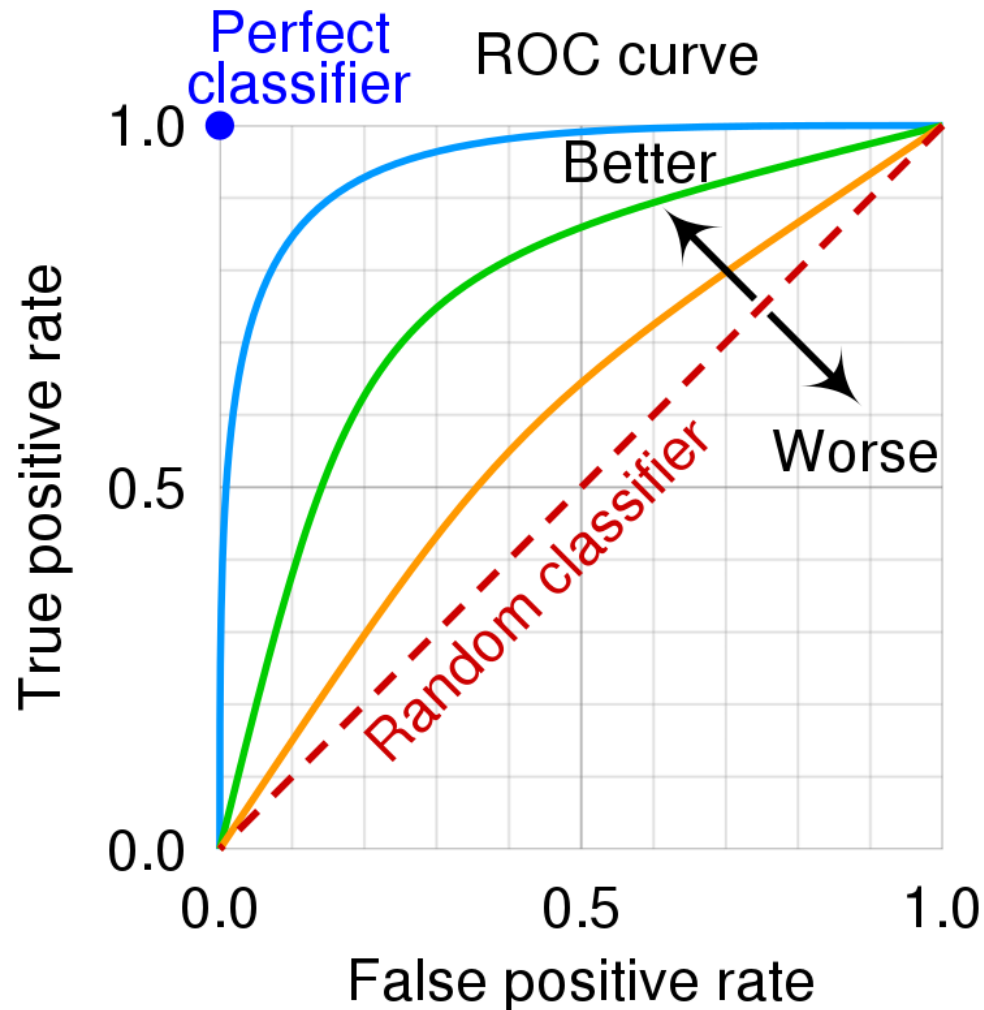
Example: Precision-recall graph



We would want to select the parameter value which results in the best precision and recall values (although what best means depends on the application).

In many cases we select the parameter values corresponding to the precision–recall pair which is closest to (1.0,1.0).

Receiver Operating Characteristic (ROC)



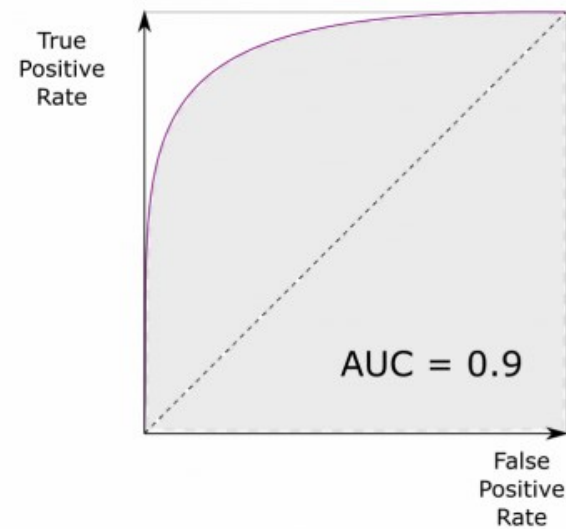
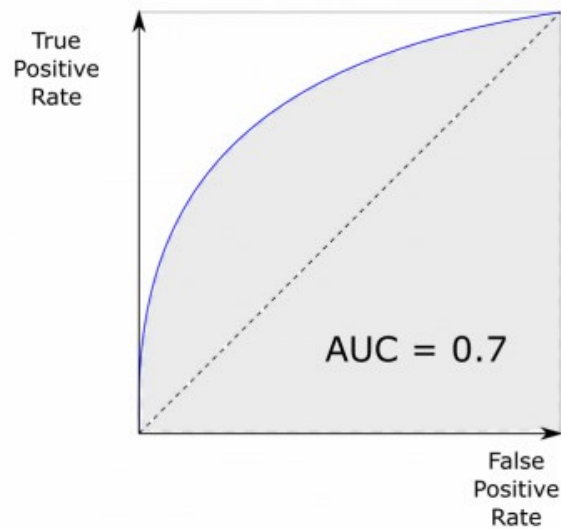
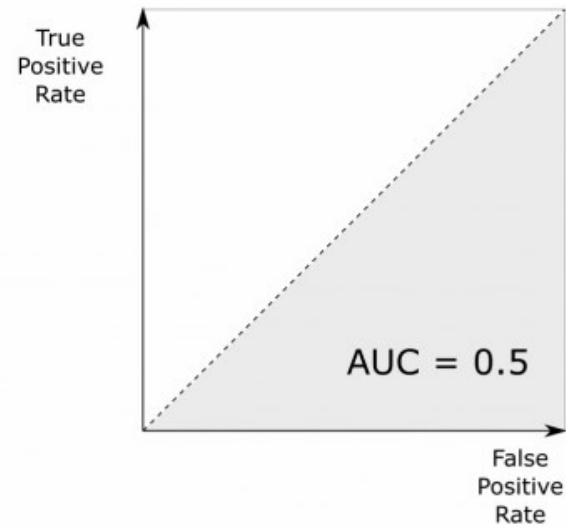
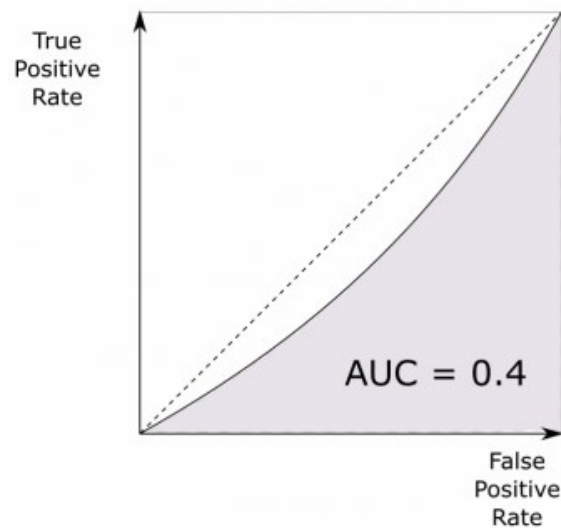
ROC depicts relative trade-offs between true positive (benefits) and false positive (costs). ROC curve is a performance measurement for the classification problems at various threshold settings.

- The best possible prediction method would yield a point in the upper left corner or coordinate (0,1) of the ROC space, representing 100% sensitivity (no false negatives) and 100% specificity (no false positives).
- A random guess would give a point along a diagonal line (the so-called line of no-discrimination) from the bottom left to the top right corners (regardless of the positive and negative base rates). A good example of random guessing is a decision by flipping coins.
- The ROC Curve is typically generated by scanning a parameter, such as a sensitivity or threshold setting. This scanning over a range is a benefit over using a single metric (such as accuracy) is that it scans range of classification thresholds – not just a single one

Area under the ROC curve (AUC)

- The ROC Curve is good for looking at the performance of a single classifier, but it is less suited to comparing the performance of different classifiers. How do we decide which is the best curve?
- In contrast, using the **area under the ROC curve (AUC)** allows different classifiers or tests to be compared. AUC of 0.5 = random guessing. AUC = 1 is perfect classifier.
- The Area Under the Curve (AUC) is **the measure of the ability of a classifier to distinguish between classes** and is used as a summary of the ROC curve. The higher the AUC, the better the performance of the model at distinguishing between the positive and negative classes.
- The area under the ROC curve (AUC) results were considered excellent for AUC values between 0.9-1.0, good for AUC values between 0.8-0.9, fair for AUC values between 0.7-0.8, poor for AUC values between 0.6-0.7 and failed for AUC values between 0.5-0.6.

Area under the ROC curve (AUC)



K-Nearest Neighbors (k-NN) example

- K-NN is known as one of the simplest nonparametric classifiers.
- K-NN is called “lazy” learning because it does not do any training at all when supplied the training data, but simply storing them. It doesn't learn a discriminative function from the training data but “memorizes” the training dataset instead.
- K-NN is a simple, supervised machine learning algorithm that can be used to solve both classification and regression problems. It's easy to implement and understand, but has a major drawback of becoming significantly slower as the size of that data in use grows.
- K-NN is an algorithm that is useful for matching a point with its closest k neighbors in a multi-dimensional space. It can be used for data that are continuous, discrete, ordinal, and categorical which makes it particularly useful for dealing with all kind of missing data.
- Purpose behind using lazy learning is that (e.g., in the K-NN which is employed in online recommendation engines like those of Netflix) the dataset gets updated with new entries on a continuous basis.

k-Nearest Neighbors (k-NN)

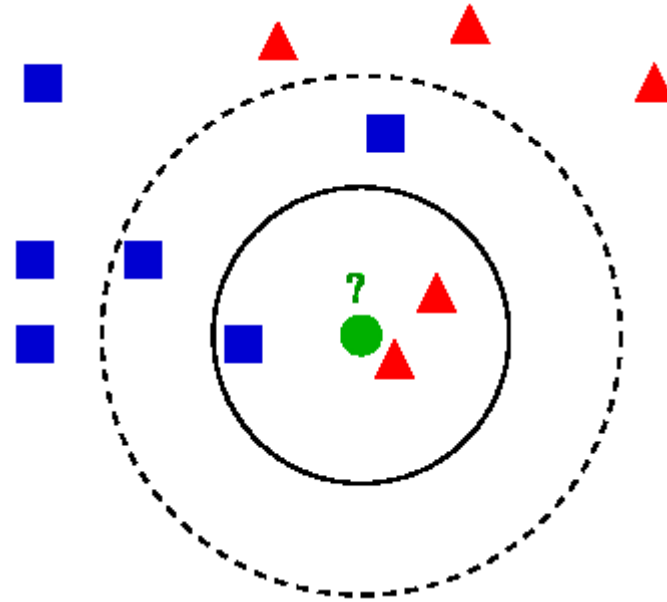
- K-NN algorithms generally do not work well with high dimensional data since, with the increasing number of dimensions, it becomes difficult to calculate the distance for each dimension. Therefore, K-NN is not suitable for high dimensional data.
- K-NN is not suitable for scalability because if the scale of features is very different then normalization is required. This is because the distance calculation done in K-NN uses feature values. When the one feature values are large than other, that feature will dominate the distance hence the outcome of the K-NN.
- To use K-NN, two important preprocessing steps are needed, 1) categorical variables must be transformed into dummy variables (e.g., factors/levels), and 2) numeric variables must be standardized/normalized.

k-Nearest Neighbors (k-NN)

The output depends on whether *k*-NN is used for **classification** or **regression**.

1. In k-NN **classification**, the output is a class membership. An object is classified by a plurality vote of its neighbors, with the object being assigned to the class most common among its *k* nearest neighbors (*k* is a positive integer, typically small).
2. In k-NN **regression**, the output is the property value for the object. This value is the average of the values of *k* nearest neighbors.

K-Nearest Neighbors (k-NN)



If k is 3, then the green dot will be assigned to a red triangle.
If k is 5, the green dot will be assigned to a blue square.

k-Nearest Neighbors (k-NN)

- It is one of the most powerful *nonparametric* classification models, first developed in 1951
- Neighborhood refers to the group of data points in geometric proximity to each other as defined by a distance metric
- The power of NN classifiers comes from the fact that the classifier we learn is relying entirely on the geometry of the training data without assuming any specific distribution. It is sensitive to the local structure of the data, but a common weighting scheme consists of giving each neighbor a weight of $1/d$, where d is the distance to the neighbor
- The most popular distance metric is the **Euclidean** distance (also called l_2 distance

$$\text{Euclidean}(x_1, x_2) = \sqrt{\sum_i (x_{1i} - x_{2i})^2}$$

k-Nearest Neighbors (k-NN)

- Another distance metric is the **Manhattan** distance or also-called **city block** distance (also called l_1 distance)
- It is the distance traveled along a grid to reach one point from the next

$$\text{Manhattan}(x_1, x_2) = \sum_i |x_{1i} - x_{2i}|$$

- A generalization of the Euclidean and Manhattan distance metrics is the **Minkowski** distance. Setting $p = 2$ gives us Euclidean distance, while setting $p = 1$ gives us Manhattan distance

$$\text{Minkowski}(x_1, x_2) = \left(\sum_i |x_{1i} - x_{2i}|^p \right)^{\frac{1}{p}}$$

Distance for numeric values

- Euclidean is a good distance measure to use if the input variables are similar in type/unit (e.g., all measured widths and heights).
- Manhattan distance is a good measure to use if the input variables are not similar in type/unit (such as age, height, etc...)
- Normalizing the data is a method that allows to give every attribute the same influence in identifying neighbors when computing certain type of distances like the Euclidean distance. We should normalize the data when the scales have no meaning and/or we have inconsistent scales like centimeters and meters.

Distance for categorical values

- Hamming distance is applicable distance for the categorical data, which is related to frequency and similarity. E.g., Hamming distance and the Weighted Hamming distance.
- Hamming distance takes all the categorical attributes and for each, count one if the value is not the same between two points. The Hamming distance is then the number of attributes for which the value was different.

4	0	1	0	0
14	1	1	1	0
HammingDistance(4,14) = 2				
4	0	1	0	0
2	0	0	1	0
HammingDistance(4,2) = 2				
14	1	1	1	0
2	0	0	1	0
HammingDistance(14,2) = 2				

Example of computing a Hamming distance

<https://medium.com/geekculture/total-hamming-distance-problem-1b74decd71c9>

Steps taken

Here is step by step on how to compute K-nearest neighbors K-NN algorithm:

1. Determine parameter K = number of nearest neighbors.
2. Calculate the distance between the query-instance and all the training samples
3. Sort the distance and determine nearest neighbors based on the K^{th} minimum distance
4. Gather the category of the nearest neighbors
5. Use simple majority of the category of nearest neighbors as the prediction value of the query instance

Summary of K-NN

- K-NN is a non-parametric and lazy learning algorithm. Non-parametric means there is no assumption for underlying data distribution, i.e., the model structure is determined from the dataset.
- It is called lazy algorithm (as opposed to eager learning) because it does not need any training data points for model generation. All training data is used in the testing phase which makes training faster and testing phase slower and costlier.
- Lazy learning refers to machine learning processes in which generalization of the training data is delayed until a query is made to the system.
- Lazy learning systems have a low or non-existent training time and a high consultation time, while eager learning systems have a high training time and a low consultation time.

Naïve Bayes example

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Continued -

Question: Given the following conditions, shall we play golf?

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

$$P(\text{Yes} | X) = P(\text{Rainy} | \text{Yes}) \times P(\text{Cool} | \text{Yes}) \times P(\text{High} | \text{Yes}) \times P(\text{True} | \text{Yes}) \times P(\text{Yes})$$

$$P(\text{Yes} | X) = 2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.00529 \rightarrow 0.2 = \frac{0.00529}{0.02057 + 0.00529}$$

Normalized

$$P(\text{No} | X) = P(\text{Rainy} | \text{No}) \times P(\text{Cool} | \text{No}) \times P(\text{High} | \text{No}) \times P(\text{True} | \text{No}) \times P(\text{No})$$

$$P(\text{No} | X) = 3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.02057 \rightarrow 0.8 = \frac{0.02057}{0.02057 + 0.00529}$$

Answer: No.

From the
frequency table
to likelihood table

Frequency Table

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3



Likelihood Table

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1



		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1



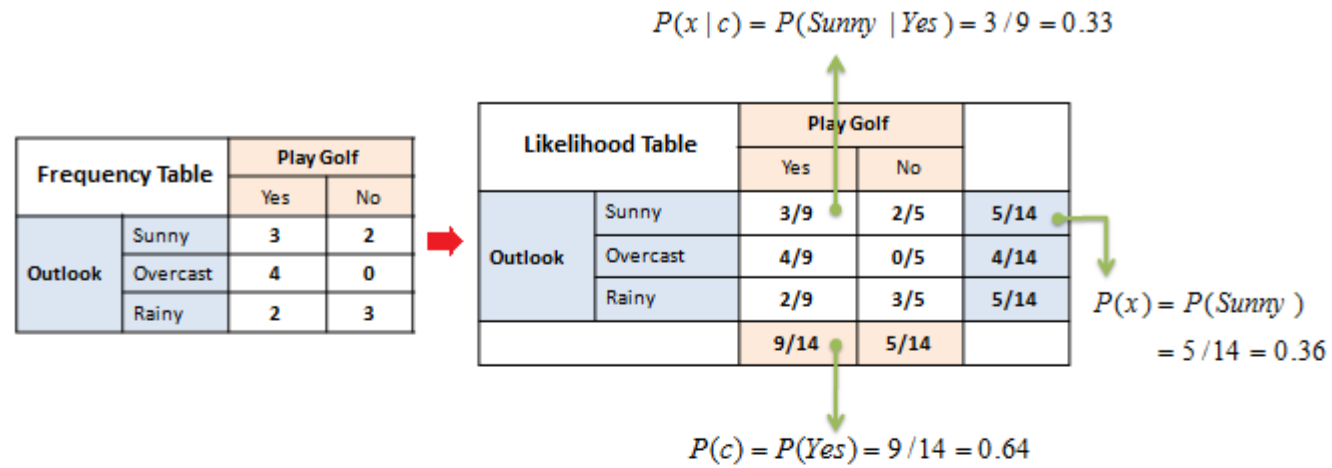
		Play Golf	
		Yes	No
Temp.	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3



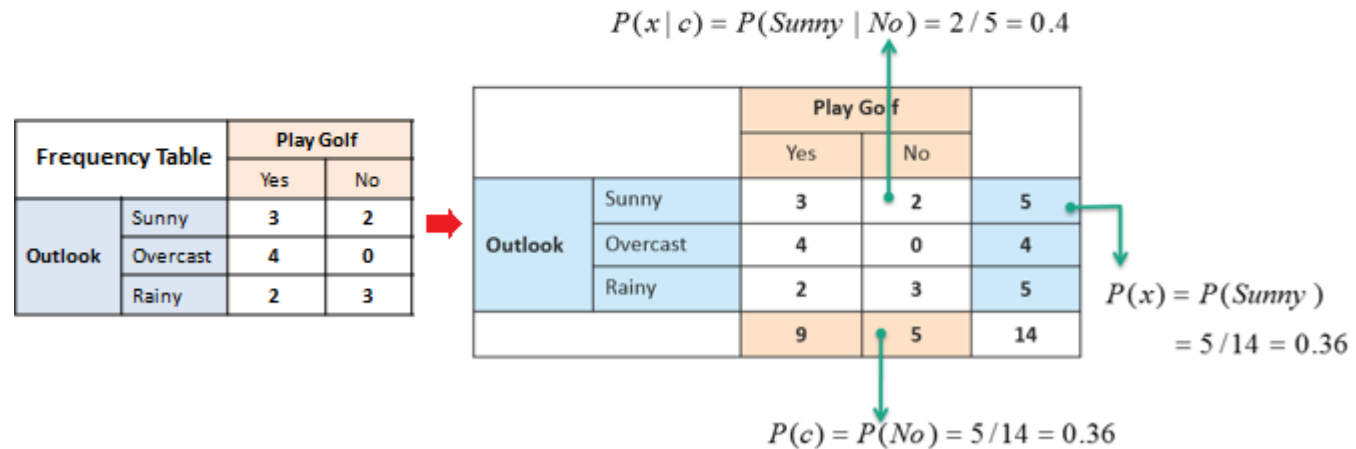
		Play Golf	
		Yes	No
Windy	False	6/9	2/5
	True	3/9	3/5

Examples of how to compute probabilities



Posterior Probability: $P(c | x) = P(\text{Yes} | \text{Sunny}) = 0.33 \times 0.64 \div 0.36 = 0.60$

↑



Posterior Probability: $P(c | x) = P(\text{No} | \text{Sunny}) = 0.40 \times 0.36 \div 0.36 = 0.40$

↑